Design using System Verilog

Assignment_3

- Toshiv Kumar, 1024060266, 3F32, ECE

Aim: Design and simulation of the serial-in serial-out and parallel-in parallel-out shift registers using basic d-flip flops.

Perform the following activities:

1.Write the System Verilog Code for D flip-flop with asynchronous reset input. Verify its operation.

Code:

Design file:

```
`timescale 1ns / 1ps


// Toshiv Kumar(NAME) , 1024060266(Roll Number)


module question_1(
    input logic clk,
    input logic rst,
    input logic d,
    output logic q
);
    always_ff @(posedge clk or posedge rst) begin
        if (rst)
            q <= 1'b0;
        else
            q <= d;
    end
```

```
endmodule
```

TB file:

```
`timescale 1ns / 1ps
module question_1_TB;
    logic clk;
    logic rst;
    logic d;
    logic q;

    question_1 dut(
        .clk(clk),
        .rst(rst),
        .d(d),
        .q(q)
    );

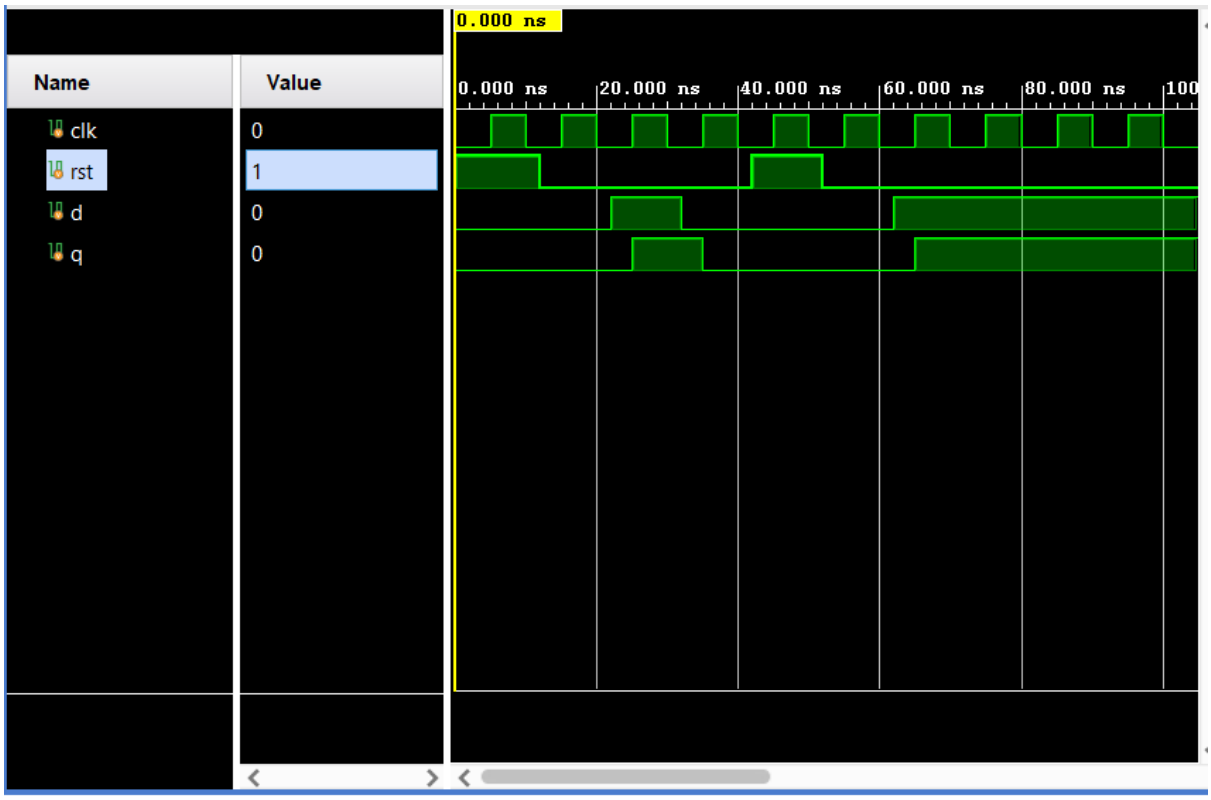
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        rst = 1;
        d = 0;
        #12 rst = 0;
        #10 d = 1;
        #10 d = 0;
```

```
        #10 rst = 1;
        #10 rst = 0;
        #10 d = 1;
        #20 $finish;
    end

    initial begin
        $monitor("time=%0t rst=%b d=%b q=%b", $time, rst, d, q);
    end
endmodule
```

Output/Screenshot:



2.Write the System Verilog Code for n-bit serial-in-serial-out shift register. Verify its operation.

Code:

Design file:

```
`timescale 1ns / 1ps

// Toshiv Kumar(NAME) , 1024060266(Roll Number)



module d_ff(
    input logic clk,
    input logic rst,
    input logic d,
    output logic q
);
    always_ff @(posedge clk or posedge rst) begin
        if (rst)
            q <= 1'b0;
        else
            q <= d;
    end
endmodule

module question_2 #(parameter n = 4)(
    input logic clk,
    input logic rst,
    input logic sin,
    output logic sout
);
    logic [n-1:0] q;

    d_ff ff0(.clk(clk), .rst(rst), .d(sin),   .q(q[0]));
```

```
    genvar i;
    generate
        for (i = 1; i < n; i = i + 1) begin : gen_ff
            d_ff ff(.clk(clk), .rst(rst), .d(q[i-1]), .q(q[i]));
        end
    endgenerate

    assign sout = q[n-1];

endmodule
```

TB:

```
`timescale 1ns / 1ps

module question_2_TB;
    logic clk;
    logic rst;
    logic sin;
    logic sout;

    question_2 #(4) dut(
        .clk(clk),
        .rst(rst),
        .sin(sin),
        .sout(sout)
    );

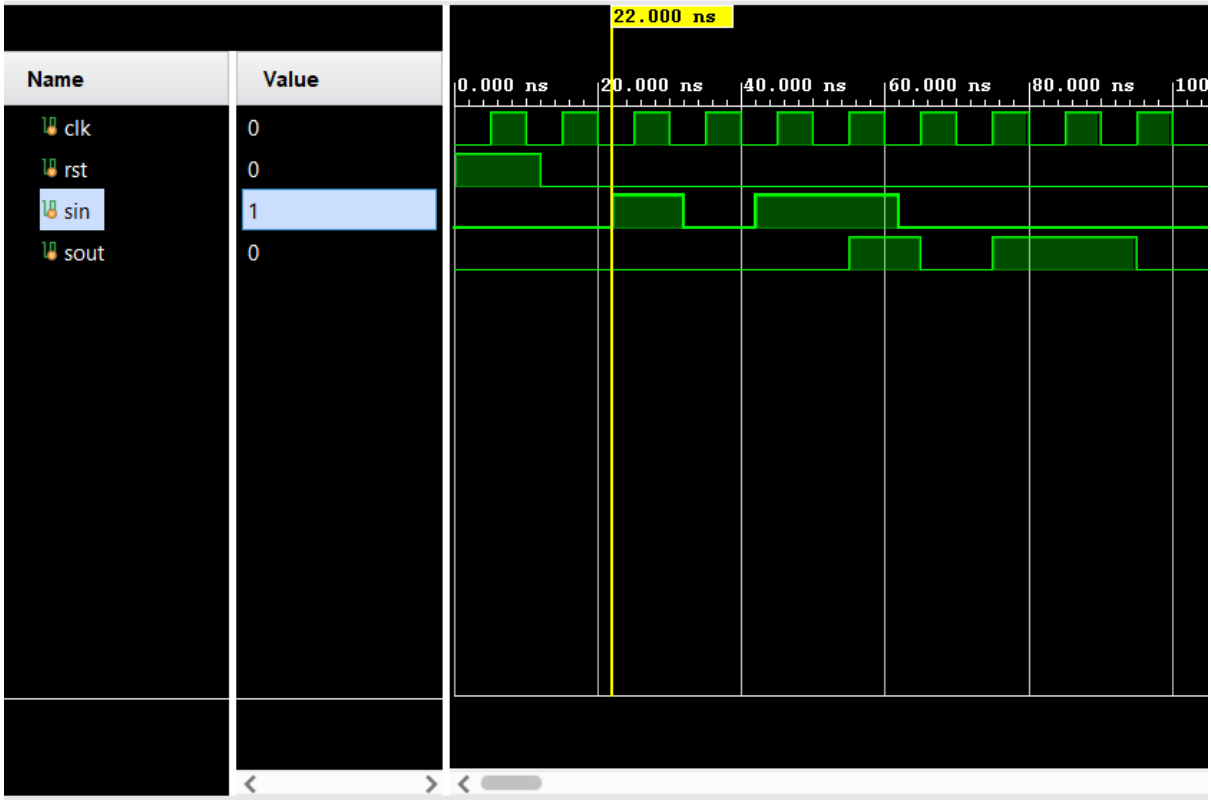
    initial begin
        clk = 0;
```

```
    forever #5 clk = ~clk;
  end

  initial begin
    rst = 1;
    sin = 0;
    #12 rst = 0;
    #10 sin = 1;
    #10 sin = 0;
    #10 sin = 1;
    #10 sin = 1;
    #10 sin = 0;
    #20 $finish;
  end

  initial begin
    $monitor("time=%0t rst=%b sin=%b sout=%b", $time, rst, sin, sout);
  end
endmodule
```

Output/Screenshot:



3.Write the System Verilog Code for n-bit parallel-in-parallel-out shift register. Verify its operation.

Code:

Design:

```
`timescale 1ns / 1ps
// Toshiv Kumar(NAME) , 1024060266(Roll Number)


module d_ff(
    input logic clk,
    input logic rst,
    input logic d,
    output logic q
);
```

```
    always_ff @(posedge clk or posedge rst) begin
        if (rst)
            q <= 1'b0;
        else
            q <= d;
    end
endmodule

module question_3 #(parameter n = 4)(
    input logic clk,
    input logic rst,
    input logic [n-1:0] din,
    output logic [n-1:0] dout
);
    genvar i;
    generate
        for (i = 0; i < n; i = i + 1) begin : gen_ff
            d_ff ff(
                .clk(clk),
                .rst(rst),
                .d(din[i]),
                .q(dout[i])
            );
        end
    endgenerate
endmodule
```

TB:

```
module question_3_TB;
    logic clk;
    logic rst;
    logic [3:0] din;
    logic [3:0] dout;

 question_3   #(4) dut(
        .clk(clk),
        .rst(rst),
        .din(din),
        .dout(dout)
    );

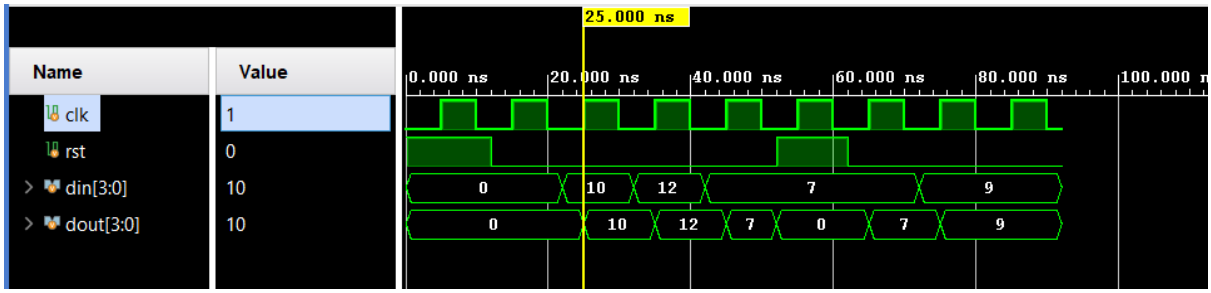
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        rst = 1;
        din = 4'b0000;
        #12 rst = 0;
        #10 din = 4'b1010;
        #10 din = 4'b1100;
        #10 din = 4'b0111;
        #10 rst = 1;
        #10 rst = 0;
        #10 din = 4'b1001;
```

```
    #20 $finish;
  end

  initial begin
    $monitor("time=%0t rst=%b din=%b dout=%b", $time, rst, din, dout);
  end
endmodule
```

Output/Screenshot:



Learnings from this Experiment:

There are many things that I learned from this experiment. First of all, I learned about D flip-flop, SISO, and pipo hardware circuits. I learned how to implement them using SystemVerilog.
At last, I learned how to verify them using SystemVerilog.